

RV32I Single-Cycle Processor Core

Design, Debug and Verification of a From-Scratch
RISC-V Datapath in Verilog

Author

Swastik Aditya Ranjan (@dr-paradox-design)
B.Tech Electrical Engineering, NIT Rourkela

Repository

[dr-paradox-design/5_Stage_Pipelined_RISC-V](#)

Status

Single-cycle core complete and verified by a self-checking regression.
5-stage pipeline: not yet started.

Toolchain

Icarus Verilog (iverilog / vvp), GTKWave
Target board (future): Digilent PYNQ-Z2

This document describes an RV32I processor core built from scratch in Verilog, following the Harris & Harris single-cycle organisation. It covers the datapath, the control unit, every module in the design, the verification strategy, a detailed post-mortem of the branch-resolution defect that made beq silently non-functional, and the plan for the 5-stage pipeline that follows.

Contents

1	Project Overview
2	Architecture and Datapath
3	Module Reference
4	Control Unit Specification
5	Instruction Set Support
6	Case Study: The Branch-Resolution Defect
7	Hardening Changes
8	Verification Strategy
9	Running the Simulation
10	Known Limitations
11	Roadmap: From Single-Cycle to 5-Stage Pipeline
A	Appendix: Top-Level Signal Reference
B	Appendix: Repository Layout

1. Project Overview

The goal of this project is to build a working RISC-V processor from first principles, in stages, with each stage verified before the next begins. The RV32I base integer ISA was chosen because it is small enough to implement completely but real enough that the resulting core runs genuine compiled code.

The work is deliberately staged:

- **Stage 1 (complete).** A single-cycle datapath: one instruction per clock, no pipeline registers, no hazards to reason about. This establishes correct instruction decode, ALU behaviour, memory access and branch resolution in the simplest possible setting.
- **Stage 2 (next).** Split the same datapath across five pipeline stages (IF, ID, EX, MEM, WB), then add the machinery that a pipeline forces you to confront: forwarding paths, a hazard-detection unit, load-use stalls and branch flushing.
- **Stage 3 (future).** Synthesise onto a Digilent PYNQ-Z2 and verify the core on real hardware rather than only in simulation.

1.1 Design Principles

- **Nothing is 'done' until a test proves it.** Every instruction the core claims to support is exercised by a committed regression program, and the testbench asserts the expected architectural state rather than relying on a human reading a waveform.
- **Test programs live outside the RTL.** The instruction memory loads `program.hex` through `$readmemh`, so changing the program never means editing or recompiling the design.
- **Every non-obvious change carries a comment.** Fixes in the RTL are marked `//FIX:` and structural or hygiene changes `//CHANGED:`, each explaining what the previous behaviour was and why it was wrong. The RTL doubles as the change log.
- **Deterministic start-up.** Both memories and the register file are explicitly zeroed, so an unwritten location reads as 0 instead of smearing X through the register file and the waveform.

2. Architecture and Datapath

The core is a classic single-cycle machine: the program counter, instruction memory, register file, ALU, data memory and write-back mux are all traversed combinationally within one clock period, and the only state elements that update on the clock edge are the PC, the register file and the data memory.

Cycles-per-instruction is exactly 1; the clock period is set by the longest path, which is the load path (PC -> instruction memory -> register file -> ALU -> data memory -> write-back mux -> register file setup).

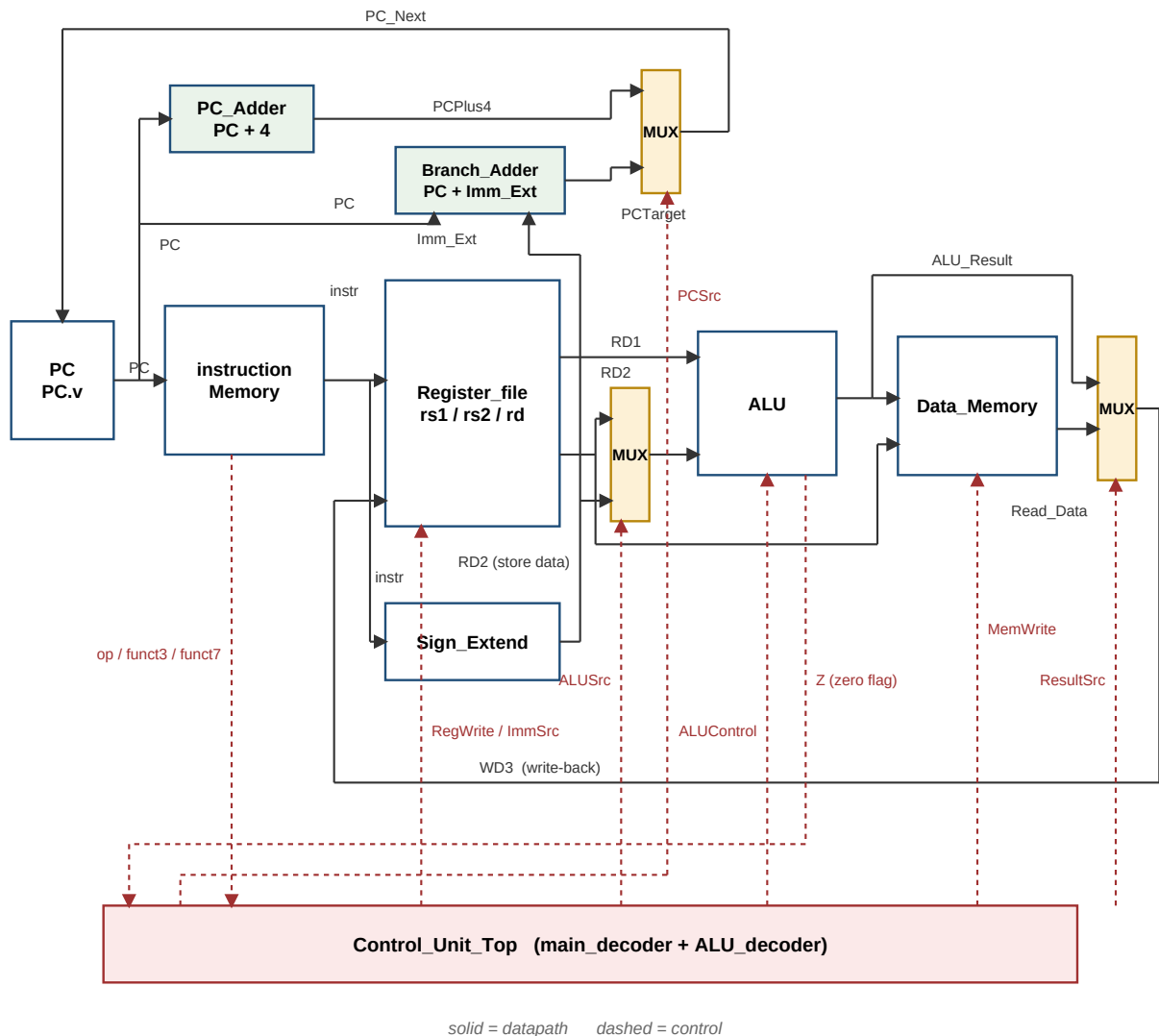


Figure 1 - Single-cycle RV32I datapath as implemented in Single_Cycle_Top.v.

2.1 Cycle Walkthrough

Within a single clock period the following happens, all combinationaly:

- **Fetch.** PC addresses **instruction_Memory**, which returns the 32-bit instruction. In parallel, **PC_Adder** computes **PC+4** and **Branch_Adder** computes **PC + Imm_Ext**.

- **Decode.** Instruction fields are sliced directly out of the fetched word: $rs1 = instr[19:15]$, $rs2 = instr[24:20]$, $rd = instr[11:7]$. `Control_Unit_Top` decodes opcode, funct3 and funct7 into every control signal, while `Sign_Extend` builds the immediate according to `ImmSrc`.
- **Execute.** The `ALUSrc` mux picks the second ALU operand: register `RD2` for R-type and branches, the sign-extended immediate for I-type, loads and stores. The ALU produces `ALU_Result` and the Z flag.
- **Memory.** For a load, `ALU_Result` is the address and `Read_Data` comes back. For a store, `MemWrite` is asserted and `RD2` is the data written at the clock edge.
- **Write-back.** The `ResultSrc` mux selects `Read_Data` for loads and `ALU_Result` otherwise; the result is written to `rd` at the clock edge when `RegWrite` is asserted.
- **Next PC.** `PCSrc` (branch opcode AND Z) selects `PCTarget` on a taken branch, otherwise `PCPlus4`. The chosen value is latched into the PC at the clock edge.

2.2 Register Conventions

`x0` is hardwired to zero: reads of address 0 return 32 'h00000000 unconditionally, and writes with `A3 == 0` are suppressed. Reset is active-low (`rst = 0` means in reset) and synchronously clears all 32 registers and the PC.

3. Module Reference

All RTL lives in `single_core/`. `Single_Cycle_Top.v` includes every leaf module, and the `testbench` includes the top level, so the whole design compiles from a single file on the iverilog command line.

Module (file)	Role	Ports
PC_Module PC.v	Program counter register. Synchronous, active-low reset to 0.	clk, rst, PC_NEXT[31:0] -> PC[31:0]
PC_Adder PC_Adder.v	Generic 32-bit adder. Instanted twice: once as PC+4, once as Branch_Adder for PC+Imm_Ext.	a[31:0], b[31:0] -> c[31:0]
instruction_Memory instruction_Memory.v	1024 x 32-bit instruction memory. Zero-filled at time 0, then loaded from program.hex via \$readmemh. Word-addressed using A[31:2].	A[31:0], rst -> RD[31:0]
Register_file Register_file.v	32 x 32-bit architectural registers. Two combinational read ports, one synchronous write port. x0 reads as 0 and is never written.	clk, rst, WE3, A1/A2/A3[4:0], WD3[31:0] -> RD1[31:0], RD2[31:0]
Sign_Extend Sign_Extend.v	Builds the 32-bit immediate for I-, S- and B-type instructions, selected by ImmSrc.	In[31:0], ImmSrc[1:0] -> Imm_Ext[31:0]
ALU ALU.v	add, sub, and, or, slt. Shares one adder for add/sub by inverting B and feeding ALUControl[0] in as carry-in. Produces Z, N, C, V flags.	A/B[31:0], ALUControl[2:0] -> Result[31:0], Z, N, C, V
main_decoder main_decoder.v	Decodes the opcode into RegWrite, MemWrite, ALUSrc, ResultSrc, ImmSrc, ALUOp, and combines the branch signal with the zero flag to form PCSrc.	op[6:0], zero -> RegWrite, MemWrite, ALUSrc, ResultSrc, ImmSrc[1:0], ALUOp[1:0], PCSrc
ALU_decoder ALU_decoder.v	Maps ALUOp plus funct3/funct7/op[5] onto the 3-bit ALUControl code.	ALUOp[1:0], funct3[2:0], funct7, op5 -> ALUControl[2:0]
Control_Unit_Top Control_Unit_Top.v	Wraps main_decoder and ALU_decoder and passes the ALU zero flag through to branch resolution.	Op[6:0], funct3[2:0], funct7[6:0], zero -> all control signals
Data_Memory Data_Mem.v	1024 x 32-bit data memory. Zero-filled at time 0. Combinational read, synchronous write on WE.	clk, rst, WE, A[31:0], WD[31:0] -> RD[31:0]
Single_Cycle_Top Single_Cycle_Top.v	Datapath integration: instantiates every module above and wires the ALUSrc, ResultSrc and PC-source muxes.	clk, rst

3.1 A Note on the Shared Adder

The ALU does not contain separate adder and subtractor hardware. `ALUControl[0]` selects between B and $\sim B$ and is simultaneously fed in as the carry-in, so a single expression covers both operations - subtraction is two's-complement addition:

```
assign not_b = ~B;
assign mux1 = (ALUControl[0] == 1'b0) ? B : not_b;
assign {Cout, sum} = A + mux1 + ALUControl[0];
```

This is why `ALUControl 000` (add) and `001` (sub) both select sum at the output mux: the distinction is already baked into how sum was computed. It is also why `beq` works at all - the branch comparison is a subtraction whose Z flag reports equality.

4. Control Unit Specification

Control is purely combinational and split in two: `main_decoder` looks only at the opcode and produces the datapath control signals plus a 2-bit `ALUOp`; `ALU_decoder` then refines `ALUOp` into a specific ALU operation using `funct3`, `funct7[5]` and `op[5]`. This two-level split keeps the opcode decode small and localises all R-type arithmetic selection in one place.

4.1 Main Decoder

Instruction	opcode	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	branch	ALUOp
R-type	0110011	1	xx	0	0	0	0	10
lw (I)	0000011	1	00	1	0	1	0	00
addi (I)	0010011	1	00	1	0	0	0	00
sw (S)	0100011	0	01	1	1	x	0	00
beq (B)	1100011	0	10	0	0	x	1	01

The final PC-source decision is one gate:

```
assign PCSrc = branch & zero;
```

A word of warning that cost real debugging time: the port on `main_decoder` is named `PCSrc`, but `Control_Unit_Top` exposes the same signal to the top level as `Branch`. It is one signal with two names, and it already includes the zero-flag term - it is not the raw 'this is a branch instruction' bit.

4.2 ALU Decoder

`concatination = {op5, funct7}`, where `funct7` here is the single bit `funct7[5]` that distinguishes add from sub. The `concatination != 2'b11` test is what stops an `addi` with a large immediate from being mistaken for a sub.

ALUOp	funct3	{op5,funct7[5]}	ALUControl	Operation	Used by
00	x	x	000	add	lw, sw, addi
01	x	x	001	subtract	beq (for the Z flag)
10	000	!= 11	000	add	add
10	000	== 11	001	subtract	sub
10	010	x	101	set less than	slt
10	110	x	011	or	or
10	111	x	010	and	and

4.3 Immediate Generation

RISC-V scrambles immediate bits across the instruction word so that each bit lands in a consistent position regardless of format, which keeps the multiplexing cheap. `Sign_Extend` unscrambles them:

```
assign Imm_Ext =
  (ImmSrc == 2'b01) ? {{20{In[31]}}, In[31:25], In[11:7]} // S-type
: (ImmSrc == 2'b10) ? {{19{In[31]}}, In[31], In[7], In[30:25], In[11:8], 1'b0} // B-type
: {{20{In[31]}}, In[31:20]}; // I-type
```

- **I-type** takes a contiguous 12-bit field from $In[31:20]$.
- **S-type** splits the 12 bits between $In[31:25]$ and $In[11:7]$, because $rs2$ occupies the middle of the word.
- **B-type** is the awkward one: the low bit is hardwired to 0 (branch targets are always even), and $In[7]$ holds bit 11 while $In[31]$ holds the sign bit. This is why a beq offset of +8 encodes as 0x00208463 rather than anything that looks like an 8.

5. Instruction Set Support

The core implements the subset of RV32I needed to run straight-line arithmetic, memory access and conditional control flow. Every listed instruction is exercised by the committed regression program - the coverage claim is checked by the test suite, not asserted by hand.

Forma t	Instruction	Semantics	Verified by
R	add rd, rs1, rs2	$rd = rs1 + rs2$	x3 = 8
R	sub rd, rs1, rs2	$rd = rs1 - rs2$	x4 = 2
R	and rd, rs1, rs2	$rd = rs1 \& rs2$	x5 = 1
R	or rd, rs1, rs2	$rd = rs1 rs2$	x6 = 7
R	slt rd, rs1, rs2	$rd = (rs1 < rs2) ? 1 : 0$	x7 = 1, x8 = 0
I	addi rd, rs1, imm	$rd = rs1 + sext(imm)$	x1 = 5, x2 = 3
I	lw rd, imm(rs1)	$rd = mem[rs1 + sext(imm)]$	x9 = 8
S	sw rs2, imm(rs1)	$mem[rs1 + sext(imm)] = rs2$	mem[0] = 8, read back
B	beq rs1, rs2, imm	if (rs1 == rs2) pc += sext(imm) else pc += 4	x10 = 1 (not taken), x11 = 0 and x12 = 7 (taken)

5.1 Not Implemented

The following RV32I instructions are deliberately out of scope for the single-cycle stage and are candidates for the pipelined version: bne, blt, bge, bltu, bgeu, jal, jalr, lui, auipc, the shift instructions (sll, srl, sra and their immediate forms), xor, sltu/sltiu, and the sub-word memory accesses (lb, lh, lbu, lhu, sb, sh). Both memories are word-addressed only; there is no byte-enable logic.

6. Case Study: The Branch-Resolution Defect

This was the single most instructive bug in the project, and it is worth documenting in full because of how it hid. The core decoded beq perfectly: the opcode was recognised, the B-type immediate was extracted correctly, and the ALU performed the subtraction. Every module looked right in isolation. But the PC never redirected, so the branch simply did nothing - and because a not-taken branch also does nothing, nothing appeared visibly broken until a test specifically required a branch to be *taken*.

6.1 Three Compounding Defects

The failure was not one bug but three, stacked such that fixing any one alone would have changed nothing observable:

- **The zero flag was thrown away.** In `Single_Cycle_Top.v` the ALU was instantiated with `.Z()` - an explicitly unconnected port. Verilog accepts this silently; it is legal to leave an output dangling.
- **The control unit hardcoded the flag to zero.** `Control_Unit_Top` had no zero input at all and passed `.zero(1'b0)` into `main_decoder`. Since `PCSrc = branch & zero`, `PCSrc` was a constant 0. This is the worst of the three: the logic was *present and correct*, just permanently starved of its input.
- **There was no branch-target path.** No adder computed `PC + Imm_Ext` and no PC-source mux existed. `PC_Module.PC_NEXT` was wired straight to `PCPlus4`, so even a correct `PCSrc` would have had nothing to select.

6.2 Before and After

Before - the entire next-PC logic:

```
PC_Module PC_Module(
    .clk(clk), .rst(rst), .PC(PC_Top),
    .PC_NEXT(PCPlus4)           // the PC could only ever advance sequentially
);

ALU ALU(
    ... .Z(),                    // zero flag discarded
    .N(), .C(), .V()
);

// inside Control_Unit_Top:
main_decoder main_decoder(
    .op(Op), .zero(1'b0),        // PCSrc = branch & 0 = always 0
    ...
);
```

After - zero flag routed through, branch adder and PC mux added:

```
wire [31:0] PCTarget, PC_Next_Top;
wire        Zero_Top;

//FIX: new PC-source mux. Branch is Control_Unit_Top's PCSrc output
//(branch opcode & ALU zero flag) - selects the branch target on a taken
//branch, otherwise PC+4. This is what actually makes beq redirect fetch.
assign PC_Next_Top = Branch ? PCTarget : PCPlus4;

PC_Module PC_Module(
    .clk(clk), .rst(rst), .PC(PC_Top),
    .PC_NEXT(PC_Next_Top)       //FIX: was PCPlus4 directly
);
```

```
//FIX: new instance. Reuses PC_Adder as a generic adder to compute the
//branch target (PC + sign-extended branch immediate).
PC_Adder Branch_Adder( .a(PC_Top), .b(Imm_Ext_Top), .c(PCTarget) );

ALU ALU(
    ... .Z(Zero_Top),          //FIX: was .Z() - now feeds Control_Unit_Top
);

Control_Unit_Top Control_Unit(
    ... .zero(Zero_Top),       //FIX: new connection
);
```

And in `Control_Unit_Top.v`, a new port replacing the hardcoded constant:

```
//FIX: added this input. Previously main_decoder's zero port was hardcoded to 1'b0
//below, so PCSrc (Branch) = branch-opcode & zero was always 0 and beq could never
//take a branch. Now driven by the ALU's real zero flag from Single_Cycle_Top.
input zero;
...
main_decoder main_decoder(
    .op(Op),
    .zero(zero),    //FIX: was hardcoded .zero(1'b0)
    ...
);
```

6.3 Lessons

- **An unconnected output port is a silent failure mode.** `.Z()` produces no warning in most flows. Reviewing the instantiation list of the top level for empty port connections is now a standard check.
- **Constants tied into control logic are worse than missing logic.** Missing logic fails to compile; a hardcoded `1'b0` compiles, simulates, and quietly disables a feature while the surrounding code looks complete.
- **Module-level correctness does not imply system-level correctness.** Every module involved was individually right. The bug lived entirely in the wiring between them, which is exactly what a top-level regression test catches and a unit test does not.
- **A test that cannot fail is not a test.** Verifying only that a not-taken branch falls through would have passed against the broken design. The regression deliberately includes a branch that must be *taken*, and the instruction it skips over writes a distinctive value (99) so a missed skip is unmistakable.

7. Hardening Changes

Beyond the branch fix, several changes were made to remove non-determinism and to make the design testable. Each is marked `//CHANGED:` in the source with the reasoning inline.

7.1 Register File: Reset Now Actually Clears

The read ports already forced 0 during reset, which *looked* like a working reset. But the underlying array was never written, so the moment `rst` deasserted, whatever X or stale values the array held became visible again. The fix clears the array synchronously:

```
always @(posedge clk) begin
    //FIX: this if/else reset branch is new. Previously only WE3 writes were handled here
    //and RD1/RD2 above merely forced *reads* to 0 during reset - the Register array itself
    //was never actually cleared, so stale values could resurface once rst deasserted.
    if (!rst) begin
        for (i = 0; i < 32; i = i + 1)
            Register[i] <= 32'h00000000;
    end
    else begin
        if (WE3 && (A3 != 5'b00000)) begin
            Register[A3] <= WD3;
        end
    end
end
end
```

7.2 Instruction Memory: Program Loaded from a File

The test program used to be a hardcoded `Mem[0] = ...` assignment inside the RTL, so trying a different program meant editing and recompiling the design. It now loads from `program.hex`:

```
//CHANGED: declared ascending [0:1023] rather than [1023:0]. Indexing is identical,
//but $readmemh warns about ambiguous fill direction on a descending memory.
reg [31:0] Mem[0:1023];

initial begin
    //zero-fill first so untouched locations read as 32'h00000000 (a harmless NOP:
    //opcode 7'b00000000 asserts neither RegWrite nor MemWrite) instead of X
    for (j = 0; j < 1024; j = j + 1)
        Mem[j] = 32'h00000000;

    $readmemh("program.hex", Mem);
end
```

Two details worth recording. First, the memory declaration was flipped from `[1023:0]` to `[0:1023]`; the indexing is identical but `$readmemh` warns about ambiguous fill direction on a descending range. Second, an all-zero word is a benign NOP in this core - opcode `7'b00000000` matches no decoder case, so neither `RegWrite` nor `MemWrite` is asserted and the instruction has no architectural effect. That is what makes zero-filling safe.

7.3 Data Memory: Zero Initialisation

Data memory started as all X, so a load from an address that had not yet been written returned X and propagated it through the register file and the entire waveform, making the trace unreadable. It is now zero-filled at time 0, matching the instruction memory.

7.4 Testbench: Self-Contained Compilation

The compile command documented in the README was broken: nothing pulled the top-level module into the testbench, so `iverilog -o out.vvp Single_Cycle_Top_TestBench.v` failed with *Unknown module type: Single_Cycle_Top*. Adding one ``include` at the top of the testbench makes the documented command work as written.

7.5 An Expected Warning

Running the simulation prints:

```
$readmemh: Not enough words in the file for the requested range [0:1023]
```

This is expected and harmless. The program is 15 instructions and the memory is 1024 words; everything past the program was already zero-filled by the loop above. A three-argument `$readmemh("program.hex", Mem, 0)` was tried and does not suppress it, so the two-argument form was kept and the warning is documented instead of being worked around.

8. Verification Strategy

Verification rests on three things: a regression program that touches every supported instruction, a self-checking testbench that asserts the resulting architectural state, and a negative test that proves the testbench can actually fail.

8.1 The Regression Program

single_core/program.hex is 15 instructions, one 32-bit word per line in hex, with // comments (which \$readmemh accepts):

```
// addr  encoding  instruction      | effect
00500093  // 0x00  addi x1, x0, 5      | x1 = 5
00300113  // 0x04  addi x2, x0, 3      | x2 = 3
002081b3  // 0x08  add  x3, x1, x2      | x3 = 8
40208233  // 0x0c  sub  x4, x1, x2      | x4 = 2
0020f2b3  // 0x10  and  x5, x1, x2      | x5 = 5 & 3 = 1
0020e333  // 0x14  or   x6, x1, x2      | x6 = 5 | 3 = 7
001123b3  // 0x18  slt  x7, x2, x1      | x7 = (3 < 5) = 1
0020a433  // 0x1c  slt  x8, x1, x2      | x8 = (5 < 3) = 0
00302023  // 0x20  sw   x3, 0(x0)       | mem[0] = 8
00002483  // 0x24  lw   x9, 0(x0)       | x9 = 8
00208463  // 0x28  beq  x1, x2, +8      | NOT taken (5 != 3)
00100513  // 0x2c  addi x10, x0, 1      | x10 = 1 (proves fall-through)
00108463  // 0x30  beq  x1, x1, +8      | TAKEN (5 == 5)
06300593  // 0x34  addi x11, x0, 99     | MUST be skipped -> x11 stays 0
00700613  // 0x38  addi x12, x0, 7      | x12 = 7 (resumes at target)
```

The program is constructed so that each check isolates one behaviour:

- sw then lw at the same address makes the store observable - a store alone writes to memory the testbench does not inspect, so it is read back into x9 instead.
- slt is tested in both directions (x7 true, x8 false) so a stuck-at output cannot pass.
- Both branch outcomes are covered. The not-taken branch is proved by x10 = 1 (the fall-through instruction ran). The taken branch is proved twice over: x11 = 0 shows the skipped instruction did *not* run, and x12 = 7 shows execution correctly resumed at the target rather than derailing.

8.2 The Self-Checking Testbench

Rather than dumping a waveform for a human to inspect, the testbench compares each register against its expected value through a check_reg task and counts mismatches:

```
task check_reg;
    input [4:0] num;
    input [31:0] expected;
    begin
        if (Single_Cycle_Top.Register_file.Register[num] != expected) begin
            $display("  FAIL: x%d = %d (expected %d)",
                    num, Single_Cycle_Top.Register_file.Register[num], expected);
            errors = errors + 1;
        end
        else
            $display("  ok  : x%d = %d", num, expected);
        end
    endtask
```

Note the != rather than !=. The case-inequality operator treats X as a distinct value, so an uninitialised register is reported as a failure; != would evaluate to X and the if would not fire, silently passing.

The task reaches into the DUT through a hierarchical reference (`Single_Cycle_Top.Register_file.Register[num]`), which is what allows architectural state to be checked without adding debug ports to the design.

8.3 Timing Analysis

The clock toggles every 50 time units, so the period is 100 and positive edges fall at $t = 100, 200, 300, \dots$ Reset deasserts at $t = 125$, before the second edge, so the first instruction retires at $t = 200$ and one more every 100 thereafter. Fifteen instructions are fetched but one is branched over, so 14 execute and the last write commits at $t = 1500$. The testbench waits until $t = 2025$ before checking - comfortably past the last write, with margin.

8.4 The Negative Test

A passing test suite only means something if the suite is capable of failing. To prove that, the branch fix was deliberately re-broken in a scratch copy of the tree: `.zero(Zero_Top)` was reverted to `.zero(1'b0)` and the regression re-run. It correctly reported:

```
FAIL: x11 = 99 (expected 0)
RESULT: FAIL - 1 check(s) failed
```

This confirms both that the fix is load-bearing and that the test has real teeth. The original working copy was untouched; the experiment ran in a throwaway directory.

8.5 Current Result

```
=== single-cycle RV32I regression (program.hex) ===
ok  : x1 = 5
ok  : x2 = 3
ok  : x3 = 8
ok  : x4 = 2
ok  : x5 = 1
ok  : x6 = 7
ok  : x7 = 1
ok  : x8 = 0
ok  : x9 = 8
ok  : x10 = 1
ok  : x11 = 0
ok  : x12 = 7
RESULT: PASS - all 12 checks passed
```

9. Running the Simulation

The design is simulated with Icarus Verilog and waveforms are viewed in GTKWave. Because the testbench includes the top level and the top level includes every leaf module, only one file needs to be named on the command line.

```
cd single_core

iverilog -o out.vvp Single_Cycle_Top_TestBench.v    # compile
vvp out.vvp                                         # run the regression
gtkwave Single_Cycle_Top_TestBench.vcd             # inspect waveforms
```

Note that vvp must be run from inside single_core/: \$readmemh("program.hex", ...) resolves the path relative to the working directory, not to the source file.

9.1 Running a Different Program

Because the program is loaded at simulation time rather than compiled in, swapping programs does not require touching the RTL. Edit single_core/program.hex - one 32-bit instruction per line in hex, // comments allowed - and update the check_reg expectations at the bottom of Single_Cycle_Top_TestBench.v to match. If the new program is longer, also extend the #1900 delay so the last write has committed before the checks run.

9.2 Useful Signals in the Waveform

Signal	Why it matters
PC_Top	Instruction address. A jump that is not +4 is a taken branch.
RD_Instr	The fetched instruction word - cross-check against program.hex.
Zero_Top	ALU zero flag. Must be 1 on the cycle a beq is taken.
Branch	PCSrc. This is the signal that was stuck at 0 before the fix.
PCTarget	Computed branch target; compare against PC_Top on the next cycle.
ALU_Result_Top	ALU output - doubles as the memory address for lw and sw.
WriteData	Value heading into the register file write port.

10. Known Limitations

- **slt is incorrect on signed overflow.** The ALU computes `slt = {31'b0, sum[31]}` - it takes only the sign bit of $A - B$. This is the standard Harris & Harris simplification and is right for the overwhelming majority of operand pairs, but when $A - B$ overflows the sign bit lies. The correct expression is `sum[31] ^ V`. The V flag is already computed by the ALU, so the fix is one XOR; it is left as-is here to stay faithful to the reference design and is recorded rather than hidden.
- **Word-addressed memory only.** Both memories index with $A[31:2]$ and have no byte-enable logic, so `lb/lh/sb/sh` cannot be supported without adding one. Misaligned accesses are silently truncated to the containing word rather than trapped.
- **Single-cycle timing.** The clock period must accommodate the full load path, so the maximum frequency is set by the slowest instruction even though most instructions do not need that much time. This is precisely the problem the pipeline stage of the project exists to solve.

- **No exceptions, no CSRs, no privileged modes.** There is no trap mechanism, no `mtvec/mcause`, and no way to signal an illegal instruction - an unrecognised opcode decodes to all-zero control and is silently ignored as a NOP.
- **No branch instructions other than `beq`.** `bne` and the ordering branches would need additional `funct3` decoding and, for the signed/unsigned comparisons, use of the N and V flags that the ALU already produces but the control unit currently ignores.
- **Simulation only.** The design has not been synthesised. Constructs such as the `initial` blocks that zero-fill memory map onto FPGA block RAM initialisation but would not exist in an ASIC flow, where an explicit reset sequence would be required.

11. Roadmap: From Single-Cycle to 5-Stage Pipeline

The single-cycle core is the reference implementation. The pipelined core must produce bit-identical architectural state for the same program - which is exactly why the regression was built first. It becomes the equivalence check for the pipeline.

11.1 Completed

- Single-cycle RV32I datapath (fetch, decode, execute, memory, write-back).
- Branch resolution: zero flag routed to control, PC-source mux, branch-target adder.
- Deterministic reset and zero-initialised memories.
- Program loading from `program.hex` via `$readmemh`.
- Self-checking regression covering every supported instruction, validated by a negative test.

11.2 Next: Pipeline Stages

The plan, in dependency order:

Step	Work	What it forces you to confront
1	Insert the four pipeline registers: IF/ID, ID/EX, EX/MEM, MEM/WB. Carry control signals forward alongside data.	Control signals must now travel with their instruction rather than being globally valid.
2	Add the forwarding unit: EX/MEM and MEM/WB results routed back to the ALU input muxes.	RAW hazards. Comparing destination registers in later stages against source registers in EX.
3	Add the hazard-detection unit with load-use stall.	The one hazard forwarding cannot fix - a load result is not available until MEM, so the dependent instruction must stall one cycle.
4	Add branch flush logic.	Control hazards. Instructions fetched after a branch that turns out to be taken must be squashed.
5	Re-run the existing regression unchanged against the pipelined core.	The final architectural state must match the single-cycle core exactly. Only the cycle count differs.
6	Synthesise for PYNQ-Z2 and verify on hardware.	Everything simulation lets you get away with: timing closure, real reset behaviour, block RAM inference.

11.3 Why the Regression Matters More Now

In a single-cycle machine, an instruction either works or it does not. In a pipeline, an instruction can work in isolation and fail only when preceded by a particular other instruction two slots earlier. That class of bug is nearly impossible to find by reading a waveform and trivial to find with a self-checking regression. Extending `program.hex` with back-to-back dependent instructions, load-use pairs and branches followed by side-effecting instructions is therefore part of the pipeline work, not an afterthought to it.

Appendix A: Top-Level Signal Reference

Every wire declared in `Single_Cycle_Top.v`, for cross-referencing against the waveform.

Signal	Width	Driven by	Meaning
PC_Top	32	PC_Module	Current instruction address
PCPlus4	32	PC_Adder	Sequential next address, PC + 4
PCTarget	32	Branch_Adder	Branch target, PC + Imm_Ext
PC_Next_Top	32	PC-source mux	Address latched into PC at the clock edge
RD_Instr	32	instruction_Memory	Fetch instruction word
RD1_Top	32	Register_file	rs1 value; first ALU operand
RD2_Top	32	Register_file	rs2 value; second ALU operand or store data
Imm_Ext_Top	32	Sign_Extend	Sign-extended immediate
SrcB_Top	32	ALUSrc mux	Second ALU operand after the mux
ALU_Result_Top	32	ALU	ALU output; memory address for lw and sw
Read_Data_Top	32	Data_Memory	Value returned by a load
WriteData	32	ResultSrc mux	Value written back to the register file
ALU_Control_Top	3	ALU_decoder	Selected ALU operation
ImmSrc	2	main_decoder	Immediate format select (I / S / B)
Zero_Top	1	ALU	ALU zero flag; drives branch resolution
RegWrite	1	main_decoder	Register file write enable
MemWrite	1	main_decoder	Data memory write enable
ALUSrc	1	main_decoder	0 = RD2, 1 = immediate
ResultSrc	1	main_decoder	0 = ALU result, 1 = memory read data
Branch	1	Control_Unit_Top	PCSrc: branch opcode AND zero flag

Appendix B: Repository Layout

<pre> . +-- docs/ +-- RISC-V Project.md +-- generate_pdf.py +-- RV32I_Single_Cycle_Core.pdf +-- single_core/ +-- PC.v +-- PC_Adder.v +-- instruction_Memory.v +-- Register_file.v +-- Sign_Extend.v +-- ALU.v +-- ALU_decoder.v +-- main_decoder.v +-- Control_Unit_Top.v +-- Data_Mem.v </pre>	<pre> Design notes / project journal Generator for this document This document Single-cycle RV32I implementation Program counter register Generic adder (PC+4 and branch target) Instruction fetch, loads program.hex 32 x 32-bit register file I / S / B immediate extension add, sub, and, or, slt + flags ALUOp/funct3/funct7 -> ALUControl Opcode -> control signals Wraps main + ALU decoders Load/store data memory </pre>
---	--

```
|  +-- Single_Cycle_Top.v           Datapath integration
|  +-- program.hex                 Test program, loaded via $readmemh
|  +-- Single_Cycle_Top_TestBench.v Self-checking testbench
|  +-- Single_Cycle_Top_TestBench.vcd/.gtkw Waveform + GTKWave session
+-- src/
|  +-- Fetch_Cycle                 Placeholder for pipeline IF stage
+-- README.md
```

Convention for reading the source: search the RTL for `//FIX:` to find every change that repaired broken behaviour, and `//CHANGED:` for structural or hygiene changes. Each comment states what the code did before and why that was wrong, so the source files carry their own history.

End of document.